()

ОТЧЁТ
О ПРАКТИЧЕСКОЙ РАБОТЕ
Практическое занятие №9
по теме:
РЕАЛИЗАЦИЯ РЕГИСТРАЦИИ И ВХОДА ПОЛЬЗОВАТЕЛЕЙ.
ХЭШИРОВАНИЕ ПАРОЛЕЙ С BCRYPT

Исполнитель НИР,
студент группы ЭФМО-01-26 _____ Акулов Роман Витальеви
подпись, дата

2026

1 Практическое занятие №9. Реализация регистрации и входа пользователей. Хэширование паролей с bcrypt

Дисциплина: Технологии индустриального программирования
Язык программирования: Go
База данных: PostgreSQL
ORM: GORM
HTTP-маршрутизатор: chi
Алгоритм хэширования: bcrypt

1.1 Цель работы

В рамках практического занятия реализую регистрацию и авторизацию пользователей. Для хранения учётных записей буду использовать PostgreSQL, а пароли перед сохранением преобразовывать в bcrypt-хэши.

В результате получу HTTP-сервис `pz9-auth` с двумя основными маршрутами: `POST /auth/register` для регистрации и `POST /auth/login` для проверки учётных данных. Также предусмотрю валидацию входящих запросов и обработку типовых ошибок.

JWT в этой работе пока не используется — его реализация предусмотрена в следующем практическом занятии.

1.2 Подготовка рабочего окружения

Для начала проверю наличие необходимых инструментов. В работе использую Go версии 1.23 или новее, PostgreSQL, Docker Desktop с Docker Compose и PowerShell.

Проверю версии Go и Docker:

```
go version

docker --version
docker compose version
```



1.3 Создание проекта и установка зависимостей

Создам рабочую директорию и инициализирую Go-модуль:

```
mkdir pz9-auth
cd pz9-auth

go mod init example.com/pz9-auth
```

После этого установлю библиотеки, которые понадобятся для маршрутизации, работы с PostgreSQL, ORM и хэширования паролей:

```
go get github.com/go-chi/chi/v5
go get gorm.io/gorm
go get gorm.io/driver/postgres
go get golang.org/x/crypto/bcrypt
```

```
PS C:\Users\romashka\practice9\pz9-auth\> go mod init example.com/pz9-auth
go: creating new go.mod: module example.com/pz9-auth
PS C:\Users\romashka\practice9\pz9-auth\> go get github.com/go-chi/chi/v5
go: added github.com/go-chi/chi/v5 v5.5.0
PS C:\Users\romashka\practice9\pz9-auth\> go get gorm.io/gorm
go: added github.com/jinzhu/inflection v1.0.0
go: added github.com/jinzhu/now v1.1.5
go: added golang.org/x/text v0.20.0
go: added gorm.io/gorm v1.31.2
PS C:\Users\romashka\practice9\pz9-auth\> go get gorm.io/driver/postgres
go: added github.com/jackc/pgpassfile v1.0.0
go: added github.com/jackc/pgservicefile v0.0.0-20240606120523-5a60cdf6a761
go: added github.com/jackc/pgx/v5 v5.10.0
go: added github.com/jackc/puddle/v2 v2.2.2
go: upgraded golang.org/x/sync v0.9.0 => v0.17.0
go: upgraded golang.org/x/text v0.20.0 => v0.29.0
go: added gorm.io/driver/postgres v1.6.3
PS C:\Users\romashka\practice9\pz9-auth\> go get golang.org/x/crypto/bcrypt
go: downloading golang.org/x/crypto v0.58.0
go: downloading golang.org/x/sync v0.24.0
go: downloading golang.org/x/text v0.43.0
go: added golang.org/x/crypto v0.58.0
go: upgraded golang.org/x/sync v0.17.0 => v0.24.0
go: upgraded golang.org/x/text v0.29.0 => v0.43.0
PS C:\Users\romashka\practice9\pz9-auth\>
```

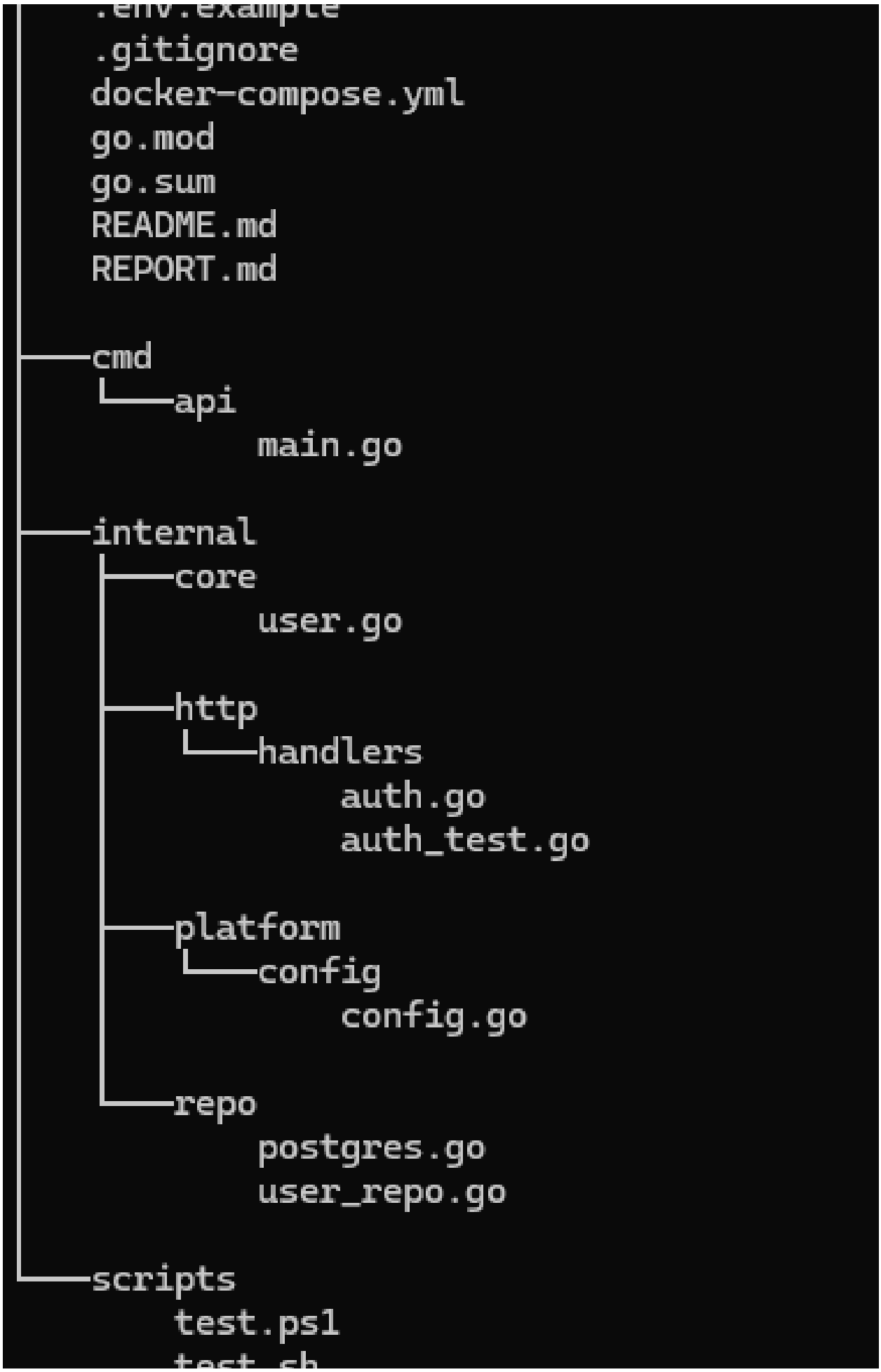
1.4 Создание структуры каталогов

Разделю приложение на несколько частей, чтобы не размещать всю логику в одном файле.

В итоге структура проекта будет выглядеть следующим образом:

```
pz9-auth/
├── cmd/
│   └── api/
│       └── main.go
├── internal/
│   ├── core/
│   │   └── user.go
│   ├── http/
│   │   └── handlers/
│   │       └── auth.go
│   ├── platform/
│   │   └── config/
│   │       └── config.go
│   └── repo/
│       ├── postgres.go
│       └── user_repo.go
└── go.mod
```

Точку входа приложения размещу в `cmd/api`. В каталоге `internal/core` будет находиться модель пользователя, в `internal/http/handlers` — HTTP-обработчики, а в `internal/repo` — функции для взаимодействия с базой данных. Настройки приложения вынесу отдельно в `internal/platform/config`.


.env.example
.gitignore
docker-compose.yml
go.mod
go.sum
README.md
REPORT.md
cmd
api
main.go
internal
core
user.go
http
handlers
auth.go
auth_test.go
platform
config
config.go
repo
postgres.go
user_repo.go
scripts
test.ps1

1.5 Настройка конфигурации приложения

Создам `internal/platform/config/config.go`. Здесь реализую чтение параметров из переменных окружения, чтобы не указывать настройки подключения непосредственно в исходном коде.

```go
package config

import (
  "fmt"
  "os"
  "strconv"

  "golang.org/x/crypto/bcrypt"
)

const defaultBcryptCost = 12
const maxRecommendedBcryptCost = 14

type Config struct {
  DBDSN      string
  BcryptCost int
  Addr       string
}

func Load() (Config, error) {
  dsn := os.Getenv("DB_DSN")
  if dsn == "" {
    return Config{}, fmt.Errorf("DB_DSN is required")
  }

  cost := defaultBcryptCost
  if value := os.Getenv("BCRYPT_COST"); value != "" {
    parsed, err := strconv.Atoi(value)
    if err != nil {
      return Config{}, fmt.Errorf("BCRYPT_COST must be an integer:
%w", err)
    }
    cost = parsed
  }

  if cost < bcrypt.MinCost || cost > maxRecommendedBcryptCost {
    return Config{}, fmt.Errorf(
      "BCRYPT_COST must be between %d and %d for this project",
      bcrypt.MinCost,
      maxRecommendedBcryptCost,
    )
  }

  addr := os.Getenv("APP_ADDR")
  if addr == "" {
    addr = ":8080"
  }
```

В конфигурации предусмотрены три параметра: `DB_DSN` отвечает за подключение к PostgreSQL, `BCRYPT_COST` задаёт вычислительную сложность bcrypt, а `APP_ADDR` определяет адрес HTTP-сервера.

Если параметр сложности не указан, приложение использует значение 12. Аналогично для HTTP-сервера предусмотрен адрес `:8080` по умолчанию. При отсутствии строки подключения приложение завершит запуск с ошибкой.

1.6 Создание модели пользователя

Теперь опишу структуру пользователя в файле `internal/core/user.go`:

```
package core

import "time"

type User struct {
  ID           int64     `gorm:"primaryKey" json:"id"`
  Email        string    `gorm:"uniqueIndex;size:255;not null"
json:"email"`
  PasswordHash string    `gorm:"size:255;not null" json:"-"`
  CreatedAt    time.Time `json:"createdAt"`
  UpdatedAt    time.Time `json:"updatedAt"`
}
```

В модели предусмотрены идентификатор, email, хэш пароля и временные метки создания и обновления записи. Для email установлен уникальный индекс, поэтому база данных не позволит сохранить несколько пользователей с одним адресом.

Отдельно стоит обратить внимание на тег `json:"-"` у `PasswordHash`. Благодаря ему значение хэша не попадёт в JSON-ответы сервера.

1.7 Подключение к PostgreSQL

Для подключения к базе данных создам `internal/repo/postgres.go`:

```go
package repo

import (
  "context"
  "fmt"

  "gorm.io/driver/postgres"
  "gorm.io/gorm"
)

func Open(ctx context.Context, dsn string) (*gorm.DB, error) {
  db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
    TranslateError: true,
  })
  if err != nil {
    return nil, fmt.Errorf("open PostgreSQL connection: %w", err)
  }

  sqlDB, err := db.DB()
  if err != nil {
    return nil, fmt.Errorf("get underlying SQL connection: %w", err)
  }

  sqlDB.SetMaxOpenConns(10)
  sqlDB.SetMaxIdleConns(5)

  if err := sqlDB.PingContext(ctx); err != nil {
    _ = sqlDB.Close()
    return nil, fmt.Errorf("ping PostgreSQL: %w", err)
  }

  return db, nil
}
```

Функция `Open` устанавливает соединение с PostgreSQL и проверяет доступность базы через `PingContext`. Дополнительно настраиваю пул SQL-соединений, ограничивая количество открытых и простаивающих подключений.

Параметр `TranslateError` позволяет GORM преобразовывать некоторые ошибки базы данных в собственные типы ошибок. В дальнейшем

это пригодится при обработке попыток повторной регистрации одного email.

1.8 Создание репозитория пользователей и миграции

Следующим этапом реализую операции с пользовательскими записями. Для этого создам `internal/repo/user_repo.go`:

```go
package repo

import (
  "context"
  "errors"

  "example.com/pz9-auth/internal/core"
  "gorm.io/gorm"
)

var (
  ErrUserNotFound = errors.New("user not found")
  ErrEmailTaken   = errors.New("email already in use")
)

type UserRepo struct {
  db *gorm.DB
}

func NewUserRepo(db *gorm.DB) *UserRepo {
  return &UserRepo{db: db}
}

func (r *UserRepo) AutoMigrate() error {
  return r.db.AutoMigrate(&core.User{})
}

func (r *UserRepo) Create(ctx context.Context, user *core.User) error
{
  if err := r.db.WithContext(ctx).Create(user).Error; err != nil {
    if errors.Is(err, gorm.ErrDuplicatedKey) {
      return ErrEmailTaken
    }
    return err
  }

  return nil
}

func (r *UserRepo) ByEmail(ctx context.Context, email string)
(core.User, error) {
  var user core.User

  err := r.db.WithContext(ctx).
    Where("email = ?", email).
    First(&user).Error
```

В репозитории предусмотрены три основных метода. `AutoMigrate` создаёт таблицу пользователей на основе структуры `User`, `Create` добавляет новую запись, а `ByEmail` выполняет поиск пользователя по адресу электронной почты.

При попытке сохранить уже существующий email возвращаю собственную ошибку `ErrEmailTaken`. Это позволит HTTP-обработчику отличать повторную регистрацию от остальных ошибок базы данных.

Для поиска использую параметризованный запрос `Where("email = ?", email)`, чтобы передаваемое значение не объединялось с SQL-выражением напрямую.

1.9 Реализация регистрации и входа

Основную логику авторизации размещу в `internal/http/handlers/auth.go`. Здесь определю структуры запросов и ответов, интерфейс хранилища пользователей, а также два обработчика.

Полное содержимое файла:

```go
package handlers

import (
  "context"
  "encoding/json"
  "errors"
  "io"
  "net/http"
  "net/mail"
  "strings"
  "unicode/utf8"

  "example.com/pz9-auth/internal/core"
  "example.com/pz9-auth/internal/repo"
  "golang.org/x/crypto/bcrypt"
)

const maxRequestBody = 1 << 20
const maxBcryptPasswordBytes = 72

type UserStore interface {
  Create(ctx context.Context, user *core.User) error
  ByEmail(ctx context.Context, email string) (core.User, error)
}

type AuthHandler struct {
  Users      UserStore
  BcryptCost int
}

type credentialsRequest struct {
  Email    string `json:"email"`
  Password string `json:"password"`
}

type responseUser struct {
  ID    int64  `json:"id"`
  Email string `json:"email"`
}

type authResponse struct {
  Status string        `json:"status"`
  User   *responseUser `json:"user,omitempty"`
}

type errorResponse struct {
  Error string `json:"error"`
```

В `Register` сначала проверяю полученный JSON, затем нормализую email и проверяю входные данные. Пароль должен содержать не менее восьми символов и не превышать ограничение bcrypt в 72 байта.

После проверки выполняю хэширование:

```
hash, err := bcrypt.GenerateFromPassword(
  []byte(input.Password),
  h.BcryptCost,
)
```

В базу передаю уже готовый хэш. Если email занят, возвращаю `409 Conflict`, а при успешной регистрации — `201 Created`.

В обработчике `Login` сначала нахожу пользователя по email, после чего сравниваю введённый пароль с сохранённым хэшем:

```
err := bcrypt.CompareHashAndPassword(
  []byte(user.PasswordHash),
  []byte(input.Password),
)
```

Если пароль не совпадает или пользователь отсутствует, возвращаю `401 Unauthorized` с одинаковым кодом `invalid_credentials`. Таким образом, по ответу API нельзя определить, зарегистрирован ли конкретный email.

Отдельно реализована `decodeJSON`: она ограничивает размер тела запроса одним мегабайтом, запрещает неизвестные поля и проверяет наличие только одного JSON-объекта.

1.10 Создание точки входа и настройка маршрутов

Теперь свяжу созданные компоненты в `cmd/api/main.go`. Здесь будет происходить загрузка конфигурации, подключение к базе данных, миграция таблицы и запуск HTTP-сервера.

```
package main

import (
  "context"
  "errors"
  "log"
  "net/http"
  "os"
  "os/signal"
  "syscall"
  "time"

  "github.com/go-chi/chi/v5"
  "github.com/go-chi/chi/v5/middleware"

  "example.com/pz9-auth/internal/http/handlers"
  "example.com/pz9-auth/internal/platform/config"
  "example.com/pz9-auth/internal/repo"
)

func main() {
  cfg, err := config.Load()
  if err != nil {
    log.Fatal("configuration:", err)
  }

  startupCtx, cancelStartup := context.WithTimeout(
    context.Background(),
    10*time.Second,
  )

  db, err := repo.Open(startupCtx, cfg.DBDSN)
  cancelStartup()

  if err != nil {
    log.Fatal("database:", err)
  }

  sqlDB, err := db.DB()
  if err != nil {
    log.Fatal("database pool:", err)
  }

  defer func() {
    if err := sqlDB.Close(); err != nil {
      log.Printf("close database connection: %v", err)
    }
```

После инициализации всех компонентов регистрирую маршруты `/auth/register` и `/auth/login`. Дополнительно оставляю `/health`, который возвращает обычный ответ `ok` и позволяет проверить, что сервер запущен.

Для HTTP-сервера также настроены middleware из chi: генерация Request ID, определение IP-адреса, журналирование запросов и восстановление после паник. Завершение работы сервера выполняется через `Shutdown`, чтобы обработать уже поступившие запросы.

1.11 Настройка PostgreSQL

Для локального запуска базы данных использую Docker Compose. В корне проекта создам `docker-compose.yml`:

```yaml
services:
  db:
    image: postgres:17-alpine
    container_name: pz9-auth-postgres
    restart: unless-stopped

    environment:
      POSTGRES_DB: pz9_auth
      POSTGRES_USER: pz9_user
      POSTGRES_PASSWORD: pz9_password

    ports:
      - "5433:5432"

    volumes:
      - pz9_auth_data:/var/lib/postgresql/data

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U pz9_user -d pz9_auth"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pz9_auth_data:
```

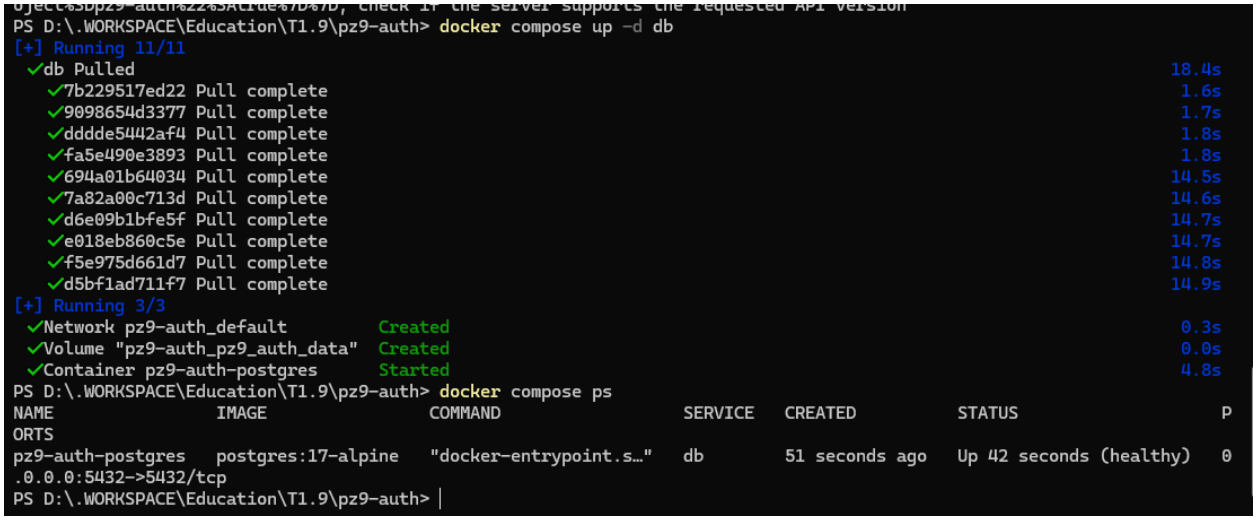
Порт 5433 на компьютере перенаправляется на стандартный порт PostgreSQL 5432 внутри контейнера. Это позволяет избежать конфликта, если локальный PostgreSQL уже использует стандартный порт.

Запущу контейнер:

```
docker compose up -d db
```

Проверю его состояние:

```
docker compose ps
```



Если вместо Docker используется локальная установка PostgreSQL, необходимо создать пользователя и базу данных. Для этого в pgAdmin открою Query Tool и выполню:

```
CREATE USER pz9_user WITH PASSWORD 'pz9_password';
CREATE DATABASE pz9_auth OWNER pz9_user;
```

При повторном запуске эти команды выполнять не нужно, если пользователь и база уже созданы.

1.12 Настройка переменных окружения и запуск приложения

Перед запуском укажу параметры подключения к PostgreSQL. В Windows PowerShell выполню:

```
$env:DB_DSN = 'postgres://pz9_user:pz9_password@127.0.0.1:5433/pz9_
auth?sslmode=disable'
$env:BCRYPT_COST = '12'
$env:APP_ADDR = ':8080'
```

Теперь проверю зависимости и выполню статический анализ:

```
go mod tidy
go vet ./...
```

Если ошибок нет, запущу приложение:

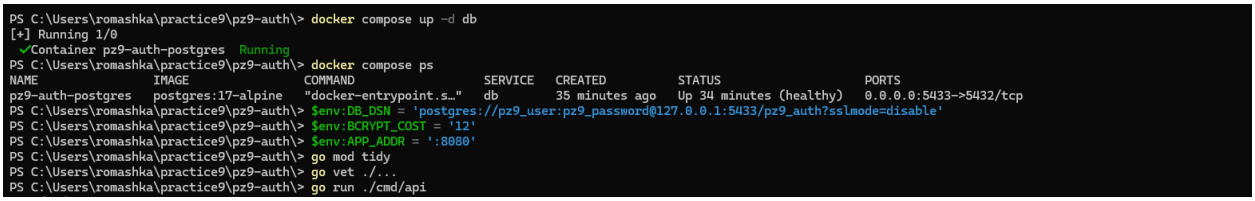
```
go run ./cmd/api
```

В терминале должно появиться сообщение о прослушивании адреса `:8080`. Для проверки открою второе окно PowerShell и отправлю GET-запрос:

```
Invoke-RestMethod -Method Get -Uri http://localhost:8080/health
```

Ожидаемый результат:

```
ok
```



1.13 Проверка регистрации и входа

После запуска сервера проверю работу обоих маршрутов. Для отправки HTTP-запросов использую `curl.exe` в PowerShell.

1.13.1 Успешная регистрация — 201 Created

Сначала зарегистрирую нового пользователя:

```
$body = @{
    email    = "user@example.com"
    password = "Secret123!"
} | ConvertTo-Json

$response = Invoke-WebRequest -Uri "http://localhost:8080/auth/
register" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body `
    -UseBasicParsing

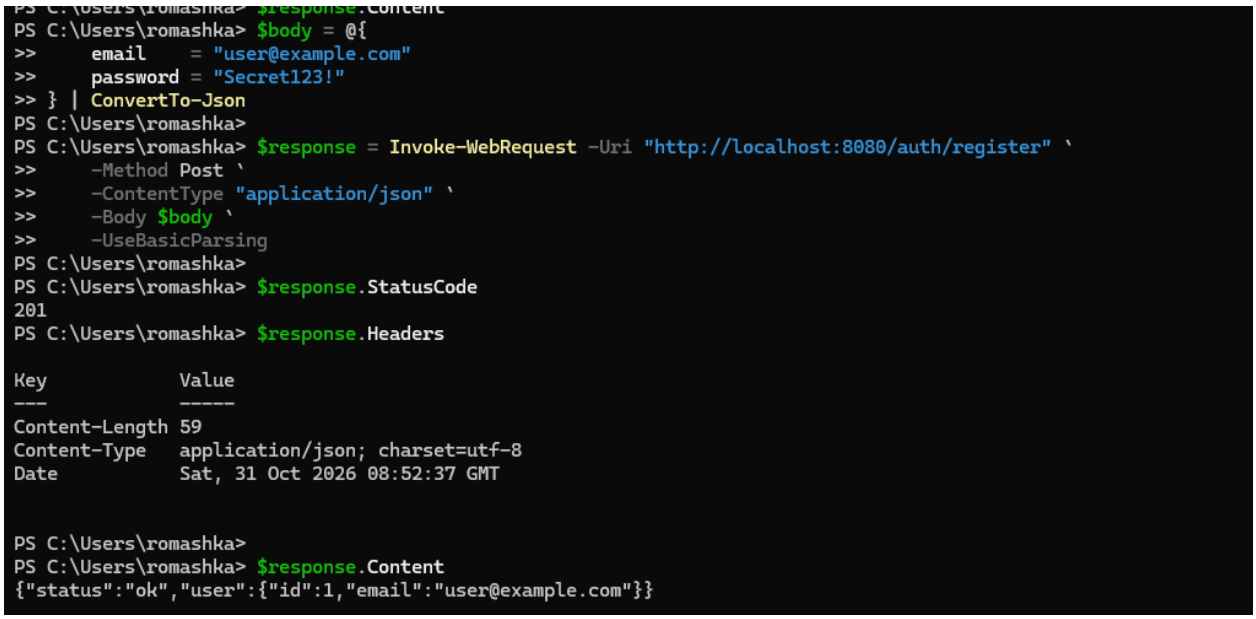
$response.StatusCode
$response.Headers

$response.Content
```

Сервер должен вернуть `201 Created` и данные созданного пользователя:

```
{
  "status": "ok",
  "user": {
    "id": 1,
    "email": "user@example.com"
  }
}
```

Идентификатор может отличаться в зависимости от состояния базы данных. Сам пароль и его хэш в ответе отсутствуют.



1.13.2 Повторная регистрация — 409 Conflict

Теперь повторю запрос с тем же email, но другим паролем:

```
$body = @{
    email    = "user@example.com"
    password = "AnotherPass123!"
} | ConvertTo-Json

try {
    $response = Invoke-WebRequest -Uri "http://localhost:8080/auth/
register" `
        -Method Post `
        -ContentType "application/json" `
        -Body $body `
        -UseBasicParsing

    "HTTP/$($response.ProtocolVersion) $([int]$response.StatusCode)
$($response.StatusDescription)"
    $response.Headers.GetEnumerator() | ForEach-Object { "$($_.Key):
$($_.Value -join ', ')" }
    ""
    $response.Content
}
catch {
    $resp = $_.Exception.Response
    "HTTP/$($resp.ProtocolVersion) $([int]$resp.StatusCode)
$($resp.StatusDescription)"
    $resp.Headers | ForEach-Object { "$($_.Key): $($_.Value -join ',
')" }
    ""
    $_.ErrorDetails.Message
}
```

Поскольку пользователь с таким адресом уже существует, получу `409 Conflict`:

```
{
  "error": "email_taken"
}
```

```
PS C:\Users\romashka> $body = @{
>>     email    = "user@example.com"
>>     password = "AnotherPass123!"
>> } | ConvertTo-Json
PS C:\Users\romashka>
PS C:\Users\romashka> try {
>>     $response = Invoke-WebRequest -Uri "http://localhost:8080/auth/register" `
>>         -Method Post `
>>         -ContentType "application/json" `
>>         -Body $body `
>>         -UseBasicParsing
>>
>>     "HTTP/$($response.ProtocolVersion) $([int]$response.StatusCode) $($response.StatusDescription)"
>>     $response.Headers.GetEnumerator() | ForEach-Object { "$($_.Key): $($_.Value -join ', ')" }
>>     ""
>>     $response.Content
>> }
>> catch {
>>     $resp = $_.Exception.Response
>>     "HTTP/$($resp.ProtocolVersion) $([int]$resp.StatusCode) $($resp.StatusDescription)"
>>     $resp.Headers | ForEach-Object { "$($_.Key): $($_.Value -join ', ')" }
>>     ""
>>     $_.ErrorDetails.Message
>> }
HTTP/1.1 409 Conflict
:
:
:

{"error":"email_taken"}
```

1.13.3 Вход с правильными данными — 200 OK

Проверю авторизацию с учётными данными, которые использовал при регистрации:

```
$body = @{
    email    = "user@example.com"
    password = "Secret123!"
} | ConvertTo-Json

$response = Invoke-WebRequest -Uri "http://localhost:8080/auth/login"
`
    -Method Post `
    -ContentType "application/json" `
    -Body $body `
    -UseBasicParsing

"HTTP/$($response.ProtocolVersion) $([int]$response.StatusCode)
$($response.StatusDescription)"
$response.Headers.GetEnumerator() | ForEach-Object { "$($_.Key):
$($_.Value -join ', ')" }
""
$response.Content
```

При успешной проверке сервер вернёт `200 OK`:

```
{
  "status": "ok",
  "user": {
    "id": 1,
    "email": "user@example.com"
  }
}
```



1.13.4 Вход с неправильным паролем — 401 Unauthorized

В последнем сценарии укажу существующий email, но передам неправильный пароль:

```
$body = @{
    email    = "user@example.com"
    password = "WrongPass123!"
} | ConvertTo-Json

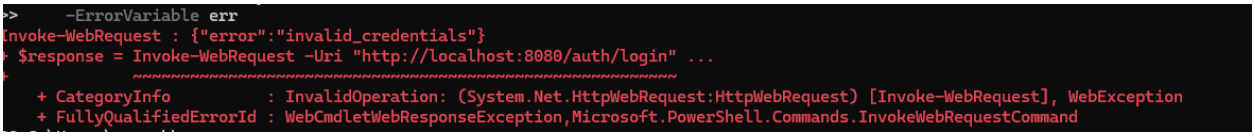
$response = Invoke-WebRequest -Uri "http://localhost:8080/auth/login"
`
    -Method Post `
    -ContentType "application/json" `
    -Body $body `
    -UseBasicParsing

"HTTP/$($response.ProtocolVersion) $([int]$response.StatusCode)
$($response.StatusDescription)"
$response.Headers.GetEnumerator() | ForEach-Object { "$($_.Key):
$($_.Value -join ', ')" }
""
$response.Content
```

Ожидаю получить `401 Unauthorized`:

```
{
  "error": "invalid_credentials"
}
```

Такой же ответ возвращается, если пользователь с указанным email отсутствует. Это сделано намеренно, чтобы не раскрывать информацию о зарегистрированных аккаунтах.



1.14 Проверка данных непосредственно в PostgreSQL

Теперь проверю, что после регистрации запись действительно появилась в базе данных.

Подключусь к контейнеру:

```
docker exec -it pz9-auth-postgres psql -U pz9_user -d pz9_auth
```

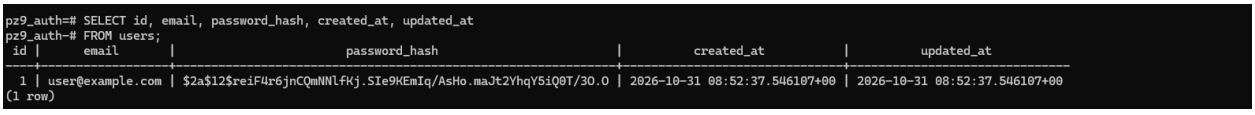
Выполню SQL-запрос:

```
SELECT id, email, password_hash, created_at, updated_at
FROM users;
```

В результате должна отобразиться созданная ранее запись. Особое внимание уделю `password_hash`: вместо исходного `Secret123!` там должна находиться строка bcrypt-хэша.

Для выхода из консоли PostgreSQL использую:

```
\q
```



1.15 Запуск автоматических тестов

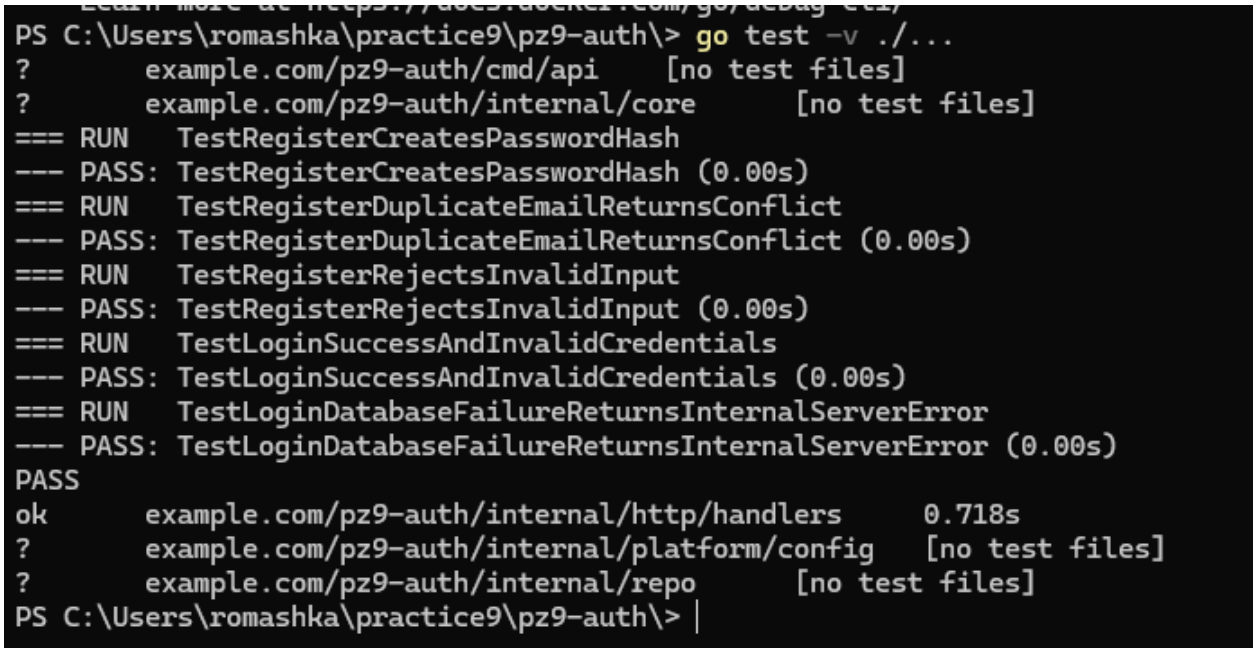
В завершение проверю проект с помощью встроенного инструментария Go:

```
go test ./...
```

Для более подробного вывода использую:

```
go test -v ./...
```

Проверки должны охватывать основные сценарии работы авторизации: создание пользователя, сохранение хэша, повторную регистрацию, валидацию данных и проверку пароля при входе.



1.16 Основные HTTP-ответы сервиса

После проверки основных сценариев получилась следующая таблица ответов API:

Маршрут	Сценарий	HTTP-статус
`POST /auth/register`	Пользователь успешно создан	`201 Created`
`POST /auth/register`	Email уже занят	`409 Conflict`
`POST /auth/register`	Некорректные данные	`400 Bad Request`
`POST /auth/login`	Правильные email и пароль	`200 OK`
`POST /auth/login`	Неверный пароль или неизвестный email	`401 Unauthorized`
`POST /auth/login`	Некорректные данные	`400 Bad Request`

Оба маршрута	Внутренняя ошибка базы данных	`500 Internal Server Error`

1.17 Ответы на контрольные вопросы

1. В чём разница между хранением пароля и хранением его хэша? Зачем нужна соль? Почему bcrypt, а не SHA-256?

При хранении пароля в открытом виде в базе находится исходная строка. Если база будет скомпрометирована, злоумышленник сразу получит пароли пользователей.

Хэширование позволяет хранить результат одностороннего преобразования. Для проверки введённого пароля приложение повторно вычисляет результат и сравнивает его с сохранённым значением.

Соль — случайное значение, добавляемое при хэшировании. Благодаря ей одинаковые пароли получают разные хэши. bcrypt самостоятельно генерирует соль и сохраняет её вместе с результатом.

SHA-256 не подходит для непосредственного хранения паролей, поскольку является быстрой универсальной хэш-функцией. bcrypt, напротив, специально предназначен для этой задачи и позволяет регулировать вычислительную сложность.

2. Что произойдёт при снижении или повышении cost у bcrypt? Как подобрать значение?

Параметр `cost` определяет сложность вычислений. При его увеличении создание и проверка хэша занимают больше времени, что усложняет массовый подбор паролей. При снижении `cost` проверка становится быстрее, но устойчивость к перебору уменьшается.

Значение подбирается с учётом производительности сервера и допустимой задержки при авторизации. В данной работе используется `cost = 12`.

3. Какие статусы и ответы должны возвращать POST /auth/register и POST /auth/login в типичных сценариях?

При успешной регистрации возвращается `201 Created`, при занятом email — `409 Conflict`, а при некорректных входных данных — `400 Bad Request`.

При успешном входе сервер возвращает `200 OK`. Если пароль неправильный или пользователь отсутствует, возвращается `401 Unauthorized`. В обоих случаях используется одинаковый код ошибки `invalid_credentials`.

4. Какие риски несут подробные сообщения об ошибках при логине?

Если сервер отдельно сообщает о существовании email и неправильном пароле, злоумышленник может проверять наличие аккаунтов. Это позволяет собирать списки зарегистрированных пользователей для дальнейших атак.

Поэтому в реализации использую одинаковые ответы для неизвестного пользователя и неправильного пароля.

5. Почему в этом практическом занятии не выдаётся токен и что изменится в ПЗ №10 (JWT)?

В текущей работе реализована только базовая проверка учётных данных. После успешного входа сервер возвращает информацию о пользователе и статус `ok`, но не создаёт токен.

В ПЗ №10 будет добавлена JWT-аутентификация. После проверки email и пароля сервер сформирует токен и подпишет его секретным ключом. Клиент сможет передавать этот токен при обращении к защищённым маршрутам, а сервер будет проверять его подпись и срок действия.